



University
Mohammed VI
Polytechnic



Deliverable #: Lab-6/ Views, Triggers, and Application development

Data Management Course
UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2025

Team Information

Team Name	StarsDB
Member 1	Nouha Talssi
Member 2	Mariam Said
Member 3	Tilamsane Wissal
Member 4	Alae Sabir
Member 5	Amina Sidki
Repository Link	https://github.com/StarsDB

1 -Introduction

This deliverable focuses on improving the simplicity of SQL queries and their optimization and performance using Views. In addition to that, we will implement Triggers to strengthen the integrity and consistency of our MNHS database. The lab also includes application development using Python to make interactions with the database more user-friendly and efficient.

NOTE: All the required files for this lab are available in the Zip file including the python application files.

2 -Methodology

To assure a structured approach to this lab, we followed these steps:

- We began by reviewing the MNHS database physical design from Lab5 to ensure the integrity of the information used in this lab.
- Next, we carefully read through the instructions of Lab6 to understand the requirements and objectives.
- We then started the work on the lab with the SQL views design following the requirements provided.
- After completing the views, we moved on to implementing the triggers to enforce data integrity and consistency.
- Then, we developed a Python application to interact with the database, using Flask for the frontend and mysql-connector to connect the application to MySQL. We then finished by implementing the required functions in the application.
- Finally, we tested the views, triggers, and application to ensure everything worked as expected.

3 Implementation & Results

For the implementation, we had three main parts: SQL Views, Triggers, and Application Development. Here are the details of each part:

3.1 SQL Views:

We designed multiple SQL views according to the lab requirements to simplify complex queries and enhance performance. Here are their implementations:

- **UpcomingByHospital view:**
 - This view aims to show all upcoming appointments scheduled in the next 14 days, by joining the tables: Appointment-ClinicalActivity-Department-Hospital, and grouping by hospital and date. This will help simplify the application code and logic by enabling direct querying of upcoming appointments by hospital.
 - SQL Code:

```

1 CREATE VIEW UpcomingByHospital AS
2 SELECT
3     H.Name AS HospitalName ,
4     C.Date AS AppDate ,
5     COUNT(A.CAID) AS ScheduledCount
6 FROM Hospital H
7 JOIN Department D ON H.HID = D.HID
8 JOIN ClinicalActivity C ON D.DEP_ID = C.DEP_ID
9 JOIN Appointment A ON C.CAID = A.CAID
10 WHERE A.Status = 'Scheduled'
11     AND C.Date >= CURDATE()
12     AND C.Date <= CURDATE() + INTERVAL 14 DAY
13 GROUP BY H.Name , C.Date ;

```

Listing 1: UpcomingByHospital View

• **DrugPricingSummary view:**

- This view provides a summary of medication prices for each hospital. It uses aggregate functions like avg, mini, max over data join Stock-Medication-Hospital. This helps in quickly retrieving pricing information without complex joins in the application layer.
- SQL Code:

```

1 CREATE VIEW DrugPricingSummary AS
2 SELECT
3     H.HID ,
4     H.Name AS HospitalName ,
5     M.MID ,
6     M.Name AS MedicationName ,
7     AVG(S.UnitPrice) AS AvgUnitPrice ,
8     MIN(S.UnitPrice) AS MinUnitPrice ,
9     MAX(S.UnitPrice) AS MaxUnitPrice ,
10    MAX(S.StockTimestamp) AS LastStockTimestamp
11 FROM Stock S
12 JOIN Hospital H ON S.HID = H.HID
13 JOIN Medication M ON S.MID = M.MID
14 GROUP BY H.HID, H.Name , M.MID , M.Name ;

```

Listing 2: DrugPricingSummary View

• **StaffWorkloadThirty view:**

- This view calculates the workload of each staff member over the past 30 days by counting the number of clinical activities they have been involved in. It joins 4 different tables and groups them by IDs. This helps offer a clear overview of appointment Status.
- SQL Code:

```

1 CREATE VIEW StaffWorkLoadThirty AS
2 SELECT

```

```

3      S.STAFF_ID ,
4      S.FullName ,
5      COUNT(A.CAID) AS TotalAppointments ,
6      SUM(CASE WHEN A.Status = 'Scheduled' THEN 1 ELSE 0
7            END) AS ScheduledCount ,
8      SUM(CASE WHEN A.Status = 'Completed' THEN 1 ELSE 0
9            END) AS CompletedCount ,
10     SUM(CASE WHEN A.Status = 'Cancelled' THEN 1 ELSE 0
11           END) AS CancelledCount
12 FROM Staff S
13 JOIN ClinicalActivity C ON S.STAFF_ID = C.STAFF_ID
14 JOIN Appointment A ON C.CAID = A.CAID
15 WHERE C.Date >= CURDATE() - INTERVAL 30 DAY
16 GROUP BY S.STAFF_ID, S.FullName;

```

Listing 3: StaffWorkloadThirty View

- **PatientNextVisit view:**

- This view retrieves the next scheduled appointment for each patient by joining Patient-Appointment-ClinicalActivity-Hospital-Department tables and filtering for the earliest upcoming date according to the current date. This helps enable the system to quickly access patients' next visits without complex queries.

– SQL Code:

```

1 CREATE VIEW PatientNextVisit AS
2 SELECT
3     P.IID ,
4     P.FullName ,
5     C.Date AS NextAppDate ,
6     D.Name AS DepartmentName ,
7     H.Name AS HospitalName ,
8     H.City
9 FROM Patient P
10 JOIN ClinicalActivity C ON P.IID = C.IID
11 JOIN Appointment A ON C.CAID = A.CAID
12 JOIN Department D ON C.DEP_ID = D.DEP_ID
13 JOIN Hospital H ON D.HID = H.HID
14 WHERE A.Status = 'Scheduled'
15        AND C.Date > CURDATE()
16        AND C.Date = (
17            SELECT MIN(C2.Date)
18            FROM ClinicalActivity C2
19            JOIN Appointment A2 ON C2.CAID = A2.CAID
20            WHERE C2.IID = P.IID
21                  AND A2.Status = 'Scheduled'
22                  AND C2.Date > CURDATE()
23        );

```

Listing 4: PatientNextVisit View

3.2 Triggers:

- **Reject double booking for a staff member Trigger:**

- This trigger prevents double booking of a staff member for the same date and time when a new appointment is inserted or updated into the Appointment table. It checks for existing appointments with the same staff ID, date, and time before allowing the insertion.
- SQL Code:

```

1 CREATE TRIGGER Reject_double_booking
2 BEFORE INSERT OR UPDATE ON Appointment
3 FOR EACH ROW
4 BEGIN
5     DECLARE V_STAFF_ID INT;
6     DECLARE V_DATE DATE;
7     DECLARE V_TIME TIME;
8     DECLARE existing_count INT;
9     SELECT STAFF_ID ,DATE,TIME
10    INTO V_STAFF_ID,V_DATE,V_TIME
11    FROM ClinicalActivity
12    WHERE CAID=NEW.CAID
13    SELECT COUNT(*) INTO existing_count
14    FROM Appointment A
15     JOIN ClinicalActivity ON A.CAID=C.CAID
16    WHERE C.STAFF_ID=V_STAFF_ID
17           AND C.Date=V_Date
18           AND C.Time=V_Time
19    IF existing_count>0 THEN
20        SIGNAL SQLSTATE 45000
21        SET MESSAGE_TEXT= DOUBLE BOOKING
22        IS NOT ALLOWED ;
23    END IF;
24 END;
```

Listing 5: Reject double booking for a staff member Trigger

- **Recompute Expense.Total when prescription lines change Trigger:**

- This trigger recompute the total expense of the linked clinical activity as the sum of the unit price of all medications in the prescription lines whenever a prescription line is inserted, updated, or deleted. It ensures that the Expense.Total field remains accurate and up-to-date.
- SQL Code:

```

1 CREATE TRIGGER new_totale
2 AFTER INSERT ON Includes
3 FOR EACH ROW
4 BEGIN
5     DECLARE ActivityID INT ;
6     DECLARE HospitalID INT ;
7     DECLARE missing INT;
```

```

8      SELECT CAID INTO ActivityID
9      FROM Prescription
10     WHERE PID=NEW.PID
11     SELECT D.HID INTO hospitalID
12     FROM ClinicalActivity CA
13     JOIN Departement D on CA.DEP_ID=D.DEP_ID
14     WHERE CA.CAID=ActivityID
15     SELECT COUNT(*)
16     INTO missing
17     FROM Includes I
18     LEFT JOIN(
19         SELECT S1.HID, S1.MID, S1.UnitPrice
20         FROM Stock S1
21         JOIN (
22             SELECT MID, HID, MAX(StockTimestamp) AS ts
23             FROM Stock
24             GROUP BY HID, MID
25         ) S2
26         ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1.
           StockTimestamp = S2.ts
27         WHERE S1.HID = hospitalID
28     ) Latest
29     ON I.MID = Latest.MID AND Latest.HID = hospitalID
30     WHERE I.PID = NEW.PID AND Latest.UnitPrice IS NULL;
31 IF missing > 0 THEN
32     SIGNAL SQLSTATE '45000'
33     SET MESSAGE_TEXT = 'Missing price for at least
           one medication cannot update Expense.';
34 END IF;
35
36 UPDATE Expense
37 SET Total = (
38     SELECT SUM(Latest.UnitPrice)
39     FROM Includes I
40     JOIN (
41         SELECT S1.HID, S1.MID, S1.UnitPrice
42         FROM Stock S1
43     JOIN (
44         SELECT MID, HID, MAX(StockTimestamp) AS
           ts
45         FROM Stock
46         GROUP BY HID, MID
47     ) S2
48     ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1
           .StockTimestamp = S2.ts
49     WHERE S1.HID = hospitalID
50     ) Latest
51     ON I.MID = Latest.MID AND Latest.HID = hospitalID
52     WHERE I.PID = NEW.PID
53 )
54 WHERE CAID = ActivityID;

```

```

55 END;
56 CREATE TRIGGER new_totale
57 AFTER DELETE ON Includes
58 FOR EACH ROW
59 BEGIN
60     DECLARE ActivityID INT ;
61     DECLARE HospitalID INT ;
62     DECLARE missing INT;
63     SELECT CAID INTO ActivityID
64     FROM Prescription
65     WHERE PID=NEW.PID
66     SELECT D.HID INTO hospitalID
67     FROM ClinicalActivity CA
68     JOIN Department D on CA.DEP_ID=D.DEP_ID
69     WHERE CA.CAID=ActivityID
70     SELECT COUNT(*)
71     INTO missing
72     FROM Includes I
73     LEFT JOIN(
74         SELECT S1.HID, S1.MID, S1.UnitPrice
75         FROM Stock S1
76     JOIN (
77         SELECT MID, HID, MAX(StockTimestamp) AS ts
78         FROM Stock
79         GROUP BY HID, MID
80     ) S2
81     ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1.
82         StockTimestamp = S2.ts
83         WHERE S1.HID = hospitalID
84     ) Latest
85     ON I.MID = Latest.MID AND Latest.HID = hospitalID
86     WHERE I.PID = NEW.PID AND Latest.UnitPrice IS NULL;
87 IF missing > 0 THEN
88     SIGNAL SQLSTATE '45000'
89     SET MESSAGE_TEXT = 'Missing price for at least
90         one medication cannot update Expense.';
91 END IF;
92
93 UPDATE Expense
94 SET Total = (
95     SELECT SUM(Latest.UnitPrice)
96     FROM Includes I
97     JOIN (
98         SELECT S1.HID, S1.MID, S1.UnitPrice
99         FROM Stock S1
100     JOIN (
101         SELECT MID, HID, MAX(StockTimestamp) AS
102             ts
103         FROM Stock
104         GROUP BY HID, MID

```

```

103         ) S2
104         ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1
           .StockTimestamp = S2.ts
105         WHERE S1.HID = hospitalID
106     ) Latest
107     ON I.MID = Latest.MID AND Latest.HID = hospitalID
108     WHERE I.PID = NEW.PID
109 )
110 WHERE CAID = ActivityID;
111 END;
112
113
114 CREATE TRIGGER new_totale
115 AFTER INSERT ON Includes
116 FOR EACH ROW
117 BEGIN
118     DECLARE ActivityID INT ;
119     DECLARE HospitalID INT ;
120     DECLARE missing INT;
121     SELECT CAID INTO ActivityID
122     FROM Prescription
123     WHERE PID=NEW.PID
124     SELCT D.HID INTO hospitalID
125     FROMClinicalActivity CA
126     JOIN Departement D on CA.DEP_ID=D.DEP_ID
127     WHERE CA.CAID=ActivityID
128     SELECT COUNT(*)
129     INTO missing
130     FROM Includes I
131     LEFT JOIN(
132         SELECT S1.HID, S1.MID, S1.UnitPrice
133         FROM Stock S1
134     JOIN (
135         SELECT MID, HID, MAX(StockTimestamp) AS ts
136         FROM Stock
137         GROUP BY HID, MID
138     ) S2
139     ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1.
       StockTimestamp = S2.ts
140     WHERE S1.HID = hospitalID
141 ) Latest
142     ON I.MID = Latest.MID AND Latest.HID = hospitalID
143     WHERE I.PID = NEW.PID AND Latest.UnitPrice IS NULL;
144 IF missing > 0 THEN
145     SIGNAL SQLSTATE '45000'
146     SET MESSAGE_TEXT = 'Missing price for at least
       one medication cannot update Expense.';
147 END IF;
148
149
150 UPDATE Expense
    
```



```

151     SET Total = (
152         SELECT SUM(Latest.UnitPrice)
153         FROM Includes I
154         JOIN (
155             SELECT S1.HID, S1.MID, S1.UnitPrice
156             FROM Stock S1
157         JOIN (
158             SELECT MID, HID, MAX(StockTimestamp) AS
159                 ts
160             FROM Stock
161             GROUP BY HID, MID
162         ) S2
163         ON S1.MID = S2.MID AND S1.HID = S2.HID AND S1
164             .StockTimestamp = S2.ts
165         WHERE S1.HID = hospitalID
166     ) Latest
167     ON I.MID = Latest.MID AND Latest.HID = hospitalID
168     WHERE I.PID = NEW.PID
169 END;

```

Listing 6: Recompute Expense.Total when prescription lines change Trigger

- **Prevent negative or inconsistent stock Trigger:**

- This trigger ensures that the stock quantity,UnitPrice for any medication in a hospital is always positive when stock records are inserted or updated.
- SQL Code:

```

1 CREATE TRIGGER Stock_BI
2 BEFORE INSERT ON Stock
3 FOR EACH ROW
4 BEGIN
5     IF NEW.Qty < 0 THEN
6         SIGNAL SQLSTATE '45000'
7         SET MESSAGE_TEXT = 'Quantity cannot be
8             negative';
9     END IF;
10
11     IF NEW.UnitPrice <= 0 THEN
12         SIGNAL SQLSTATE '45000'
13         SET MESSAGE_TEXT = 'Unit price must be
14             positive';
15     END IF;
16
17     IF NEW.ReorderLevel < 0 THEN
18         SIGNAL SQLSTATE '45000'
19         SET MESSAGE_TEXT = 'Reorder level cannot be
20             negative';
21     END IF;
22 END;

```

```

20
21
22
23 CREATE TRIGGER Stock_BU
24 BEFORE UPDATE ON Stock
25 FOR EACH ROW
26 BEGIN
27
28     IF NEW.Qty < 0 THEN
29         SIGNAL SQLSTATE '45000'
30         SET MESSAGE_TEXT = 'Quantity cannot be
31                               negative';
32     END IF;
33
34     IF NEW.UnitPrice <= 0 THEN
35         SIGNAL SQLSTATE '45000'
36         SET MESSAGE_TEXT = 'Unit price must be
37                               positive';
38     END IF;
39
40     IF NEW.ReorderLevel < 0 THEN
41         SIGNAL SQLSTATE '45000'
42         SET MESSAGE_TEXT = 'Reorder level cannot be
43                               negative';
44     END IF;
45
46     IF NEW.Qty < OLD.Qty AND NEW.Qty < 0 THEN
47         SIGNAL SQLSTATE '45000'
48         SET MESSAGE_TEXT = 'Cannot decrease quantity
49                               below zero';
50     END IF;
51 END;

```

Listing 7: Prevent negative or inconsistent stock Trigger

- **Protect referential integrity on patient delete Trigger:**

- This trigger prevents the deletion of a patient record if there are any associated clinical activities in the database.
- SQL Code:

```

1 CREATE TRIGGER Patient_BD
2 BEFORE DELETE ON Patient
3 FOR EACH ROW
4 BEGIN
5     IF EXISTS (
6         SELECT 1
7         FROM ClinicalActivity
8         WHERE IID = OLD.IID
9     ) THEN
10         SIGNAL SQLSTATE '45000'

```

Listing 8: Protect referential integrity on patient delete Trigger

– **LIMIT1:** Attempt to insert negative stock quantity:

```
1 INSERT INTO Stock (MID, Qty, UnitPrice, ReorderLevel)
2 VALUES (5, -3, 20, 5);
3 --example two:
4 UPDATE Stock
5 SET Qty = Qty - 10
6 WHERE MID = 3;
```

Listing 11: Prevent negative or inconsistent stock

– **LIMIT2:** Protect referential integrity on patient deletion:

```
1 DELETE FROM Patient
2 WHERE IID = 3;
```

Listing 12: Protect referential integrity on patient delete

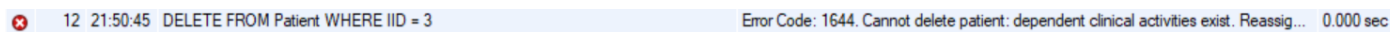


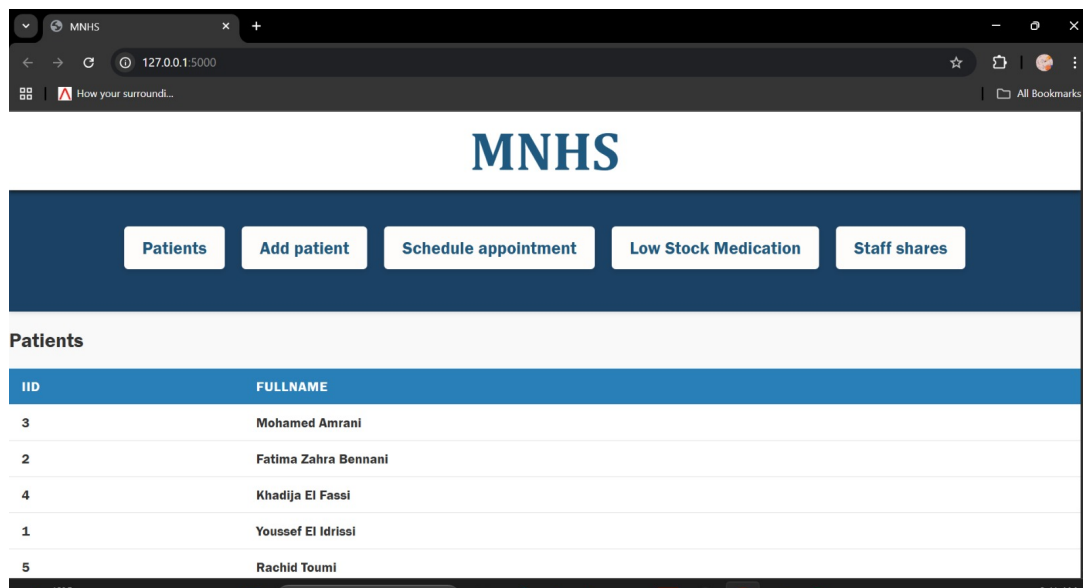
Figure 3: result of LIMIT2

3.3 Application Development:

To bring our application to life, we began by studying the starter code provided by the professor. After that, we searched for a beginner-friendly framework to help us build the backend of the application and decided to use Flask.

We structured our project into two main files:

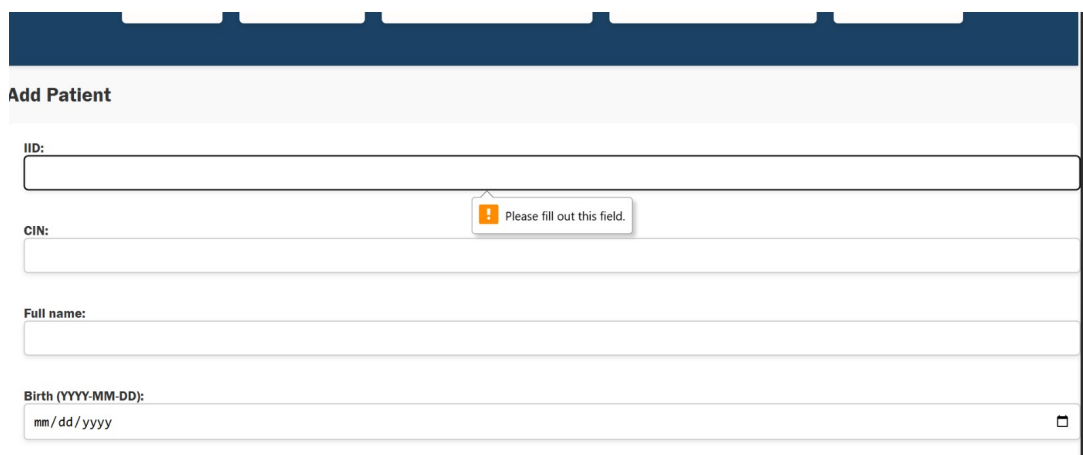
- `db_helpers.py`: contains all the functions that interact directly with the database, whether it's inserting, updating, deleting, or retrieving data.
- `MNHS_app.py`: contains the Flask routes and functions that connect the user interface to the logic in `db_helpers.py`, effectively linking the frontend to the database through Flask.
- For the frontend, we used HTML and CSS, building on the knowledge we already had in these technologies.
- Here are some screenshots of the application tests:



The screenshot shows a web browser window with the URL 127.0.0.1:5000. The page title is "MNHS". Below the title is a navigation bar with five buttons: "Patients", "Add patient", "Schedule appointment", "Low Stock Medication", and "Staff shares". The "Patients" button is selected. Below the navigation bar is a section titled "Patients" containing a table with two columns: "IID" and "FULLNAME".

IID	FULLNAME
3	Mohamed Amrani
2	Fatima Zahra Bennani
4	Khadija El Fassi
1	Youssef El Idrissi
5	Rachid Toumi

Figure 4: List of the last 20 patients ordered by last name(the data base used contains less than 20 patients therefore they were all displayed)



The screenshot shows the "Add Patient" form. It has four input fields: "IID:", "CIN:", "Full name:", and "Birth (YYYY-MM-DD):". The "CIN:" field is highlighted with a red border and a warning icon, with a message "Please fill out this field." below it. The "Birth" field has a date picker icon.

Figure 5: The form used to add a patient to the data base . The form cannot be submitted unless all the required fields are filled

Birth (YYYY-MM-DD):
mm/dd/yyyy

Sex

Blood group:

- O+
- A+
- A-
- B+
- B-
- O+
- O-

Add

Figure 6: In order to insure correct input in terms of format we used select menus

Add Patient

IID:
7

CIN:
MC330188

Full name:
Najat Chakir

Birth (YYYY-MM-DD):
11/05/2025

Figure 7: Example for adding a patient

Birth (YYYY-MM-DD):
11/05/2025

Sex
Female

Blood group:

O+
A+
A-
B+
B-
O+
O-

Add

Figure 8: Example for adding a patient

127.0.0.1:5000/low_stock

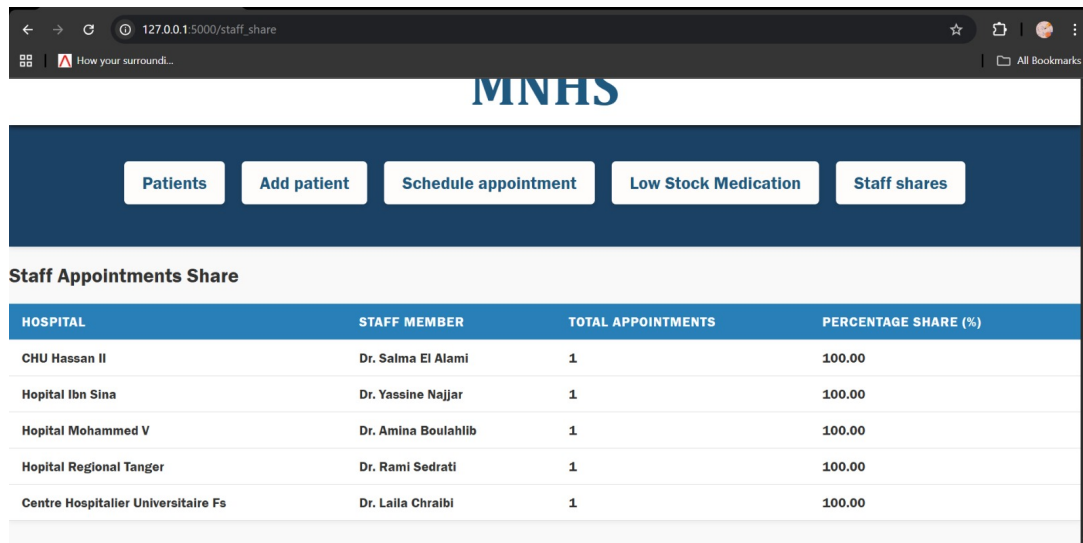
MNHS

Patients Add patient Schedule appointment Low Stock Medication Staff shares

Low Stock Medications

HOSPITAL	MEDICATION	STOCK	REORDER LEVEL
CHU Hassan II	Paracetamol	5	10
CHU Hassan II	Amoxicillin	0	10

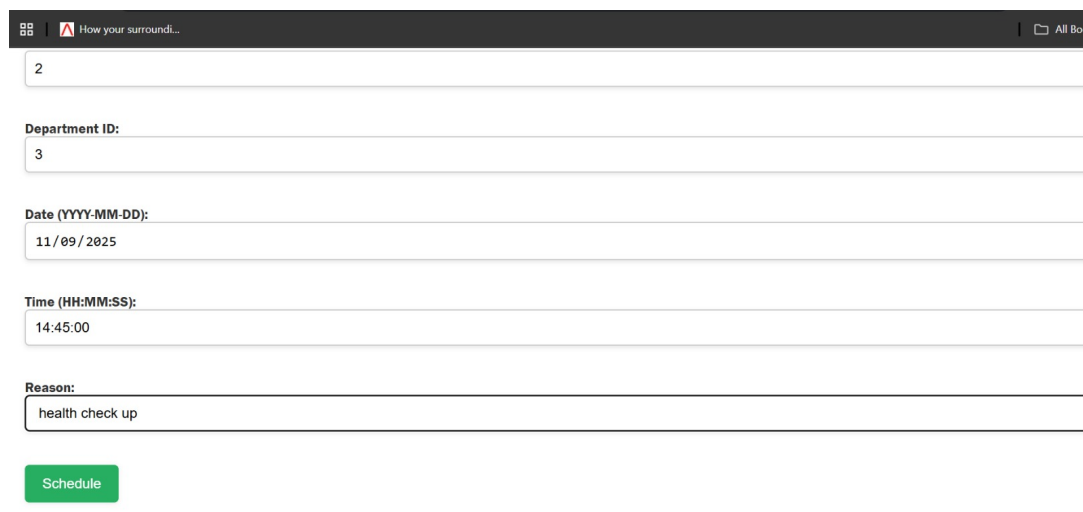
Figure 9: List of medications that are below reorder level



The screenshot shows the MNHS web application interface. At the top, there is a navigation bar with the MNHS logo and several buttons: 'Patients', 'Add patient', 'Schedule appointment', 'Low Stock Medication', and 'Staff shares'. Below this, the 'Staff Appointments Share' section is displayed, featuring a table with the following data:

HOSPITAL	STAFF MEMBER	TOTAL APPOINTMENTS	PERCENTAGE SHARE (%)
CHU Hassan II	Dr. Salma El Alami	1	100.00
Hopital Ibn Sina	Dr. Yassine Najjar	1	100.00
Hopital Mohammed V	Dr. Amina Boulahlib	1	100.00
Hopital Regional Tanger	Dr. Rami Sedrati	1	100.00
Centre Hospitalier Universitaire Fs	Dr. Laila Chraibi	1	100.00

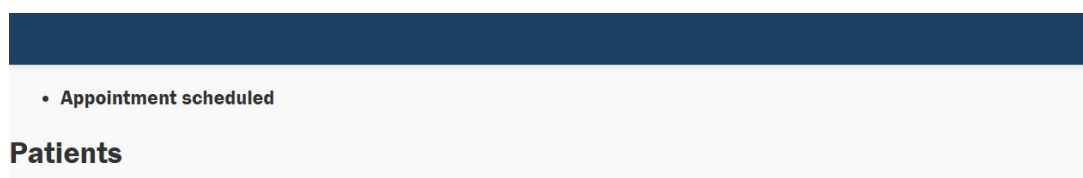
Figure 10: Sorted list of staff members shares and number of appointments



The screenshot shows the appointment scheduling form. It includes the following fields and values:

- Department ID:** 3
- Date (YYYY-MM-DD):** 11/09/2025
- Time (HH:MM:SS):** 14:45:00
- Reason:** health check up
- Schedule button:** A green button labeled 'Schedule'.

Figure 11: The added Patient appears now on the Patient list as well



The screenshot shows a confirmation message for scheduling an appointment. It features a dark blue header bar with the text 'Appointment scheduled' in white. Below this, the word 'Patients' is displayed in a bold, dark font.

Figure 12: The confirmation message for scheduling an appointment

• Appointment scheduled	
Patients	
IID	FULLNAME
3	Mohamed Amrani
2	Fatima Zahra Bennani
7	Najat Chakir
4	Khadija El Fassi
1	Youssef El Idrissi
5	Rachid Toumi

Figure 13: Example for scheduling an appointment :Input types were used in the HTML to insure correct input patterns

3.3.1 Functions

Here is the SQL code for the mini tasks we were assigned:

- Command low_stock():

```

1 SELECT
2     H.HID ,
3     H.Name AS Hospital ,
4     M.MID ,
5     M.Name AS Medication ,
6     S.ReorderLevel ,
7     IFNULL(S.Qty, 0) AS Qty
8 FROM Medication M
9 LEFT JOIN Stock S ON S.MID = M.MID
10 LEFT JOIN Hospital H ON H.HID = S.HID
11 WHERE IFNULL(S.Qty, 0) < S.ReorderLevel;

```

Listing 13: list_patients Function

- staff_share():

```

1 SELECT
2     S.STAFF_ID ,
3     ANY_VALUE(S.FullName) AS Staff ,
4     H.HID ,
5     ANY_VALUE(H.Name) AS Hospital ,
6     COUNT(A.CAID) AS TotalAppointments ,
7     ROUND(
8         COUNT(A.CAID) / dep_totals.total_per_dep * 100, 2
9     ) AS SharePercent
10 FROM Staff S
11 LEFT JOIN ClinicalActivity CA ON CA.STAFF_ID = S.STAFF_ID
12 LEFT JOIN Department D ON D.DEP_ID = CA.DEP_ID
13 LEFT JOIN Hospital H ON H.HID = D.HID

```

```

14     LEFT JOIN Appointment A ON A.CAID = CA.CAID
15     LEFT JOIN (
16         SELECT DEP_ID, COUNT(*) AS total_per_dep
17         FROM ClinicalActivity
18         GROUP BY DEP_ID
19     ) AS dep_totals ON dep_totals.DEP_ID = CA.DEP_ID
20     GROUP BY S.STAFF_ID, H.HID, dep_totals.total_per_dep
21     ORDER BY H.HID, SharePercent DESC

```

Listing 14: staff_share Function

3.3.2 AI use in this lab:

We used AI to assist in building the backend of the application by taking the professor's code as a reference, then modifying it and integrating the Flask framework. AI also helped us understand the app's code and most of the HTML. Although much of the HTML and CSS was initially AI-generated, we manually adjusted it according to the team's display preferences (for example, select menus, input types, and input constraints were not AI-generated). For the smaller tasks we were assigned, no AI was used. We followed the format of the functions predefined by the professor and used AI to understand how Flask was applied to them, which helped us grasp the overall implementation process.

4 Discussion

During this lab, our group encountered several challenges that required discussion and teamwork to resolve. Here are some of the main challenges we faced:

- **Understanding Requirements:** Since it is our first time designing a web application connected to a database, we had to spend extra time to understand this lab's requirements and how to implement them effectively. We held group discussions to clarify any ambiguities and ensure everyone was on the same page.
- **SQL Implementation:** We initially found some challenges understanding the concept of views and triggers and how to properly implement them using SQL, especially when dealing with multiple joins and aggregate functions. We then discussed each other's code to identify errors and optimize the queries for better performance.
- **Flask Use:** Using Flask for the first time was a significant problem at first. We had to learn the framework's basics and how to integrate it with our MySQL database. We also used AI tools as stated earlier to help us understand the code structure and implementation.
- **Connecting Frontend and Backend:** Ensuring smooth communication between the frontend and backend was also a big challenge for us. Especially when the connection kept crashing due to incorrect credentials or misconfigured settings. We had to carefully check our connection and environment variables to resolve these issues.
- **Testing:** We faced several bugs during the implementation of the application. Especially with the use of AI-generated code, which needed careful review and

debugging to ensure it's functionality according to our requirements. We tested each feature thoroughly and worked together to fix any issues that arose.

5 Conclusion

In this deliverable, we successfully implemented SQL Views and Triggers to enhance performance and data integrity in our MNHS database. Additionally, we developed a web application using Flask to provide a user-friendly interface for interacting with the database. We faced several challenges along the way, but through discussions and teamwork, we were able to overcome them and finish the lab requirements.